

DEVELOPMENT ENVIRONMENT

& Capacitor
Android Packing

نظام المدرسة — مدرسة حسين أبوهامد الأهلية

ABOUT THIS GUIDE

How the offline school-management system is wired — Node.js, Express and SQLite on the desktop with a vanilla-JS Arabic RTL front-end — how to run it in the browser, and how to wrap it into an Android APK with Capacitor.



PROJECT

School_system

STACK

Node 24 · Express · SQLite

SERVES ON

localhost:6000

ANDROID ID

com.school.nidham

01 The Environment

What is on this machine, and where it lives

Everything happens inside **D:\DEV** in Git Bash on Windows. The active project is **School_system** — the only one with a ready `capacitor.config.ts`.

PROJECTS IN D:\DEV

FOLDER	WHAT IT IS
School_system/	Active project — offline Arabic school management (Express + SQLite, vanilla-JS RTL front-end)
Chromaflow/	Next.js app — already has a working android/ Capacitor build
Enjazatec app/	Next.js + Prisma project (Bun)
Labatak/ · open design/	Other design and project folders
Android-build-tools/	Empty — the SDK actually lives at %LOCALAPPDATA%\Android\Sdk

TOOLCHAIN — INSTALLED VS MISSING

TOOL	STATUS	DETAIL
Node.js + npm	READY	Node v24.20.0, npm 11.19.0
Android SDK	READY	C:\Users\Ahmed\AppData\Local\Android\Sdk — platforms android-37, build-tools 36.0.0
ANDROID_HOME	MISSING	Environment variable must be set before Gradle builds — \$04
JDK (Java)	MISSING	JDK 21 needed for Gradle / Capacitor 7+ — \$04
Capacitor packages	MISSING	Not yet in School_system — \$05 installs them

SHELL CONVENTION

All commands in this guide use **Git Bash** syntax. Where CMD differs (`set PORT=6060 && npm start` instead of `PORT=6060 npm start`), it is noted inline.

02 How the App Works in the Browser

Architecture — who serves what, to whom

Browser — Chrome / Edge

UI only · opens **http://localhost:6000** · vanilla JS, Arabic RTL, hash router

client/ — static files

index.html · js/app.js ·
screens/ · styles/ · bundled
Cairo font

GET / → static files



fetch /api/* → JSON envelope

{ success, data, error } — every endpoint answers in this shape

Express server — server/app.js : 6000

routes/ students · attendance · homework · payments · import · classes ·
dashboard · export



prepared statements · better-sqlite3

data/school.db — SQLite file

Created by **npm run setup** · 24 classes seeded · students, attendance,
homework, payments

THE ONE FACT THAT SHAPES EVERYTHING

The **API and the database live in Node on your PC** — the browser is only the UI. Inside an Android APK there is no Node.js and no Express, so every `/api` call would fail. This is resolved in §06 — read it before shipping.

✓ Fully offline

All data lives in one local SQLite file; the Cairo Arabic font is bundled — zero internet needed at runtime.

✓ Two languages, one rule

UI language is Arabic (RTL); code language is English — files, variables, comments.

03 Running in the Browser

From clone to working app in three commands

```
# first time only
$ cd /d/DEV/School_system
$ npm install          # express + better-sqlite3
$ npm run setup        # create data/school.db + seed the 24 classes

# every day
$ npm start            # → http://localhost:6000
$ npm run seed:demo    # optional: Arabic demo data for testing
```

Then open **http://localhost:6000**. The database, the server and the UI are all local — nothing leaves the machine.

1 Port 6000 is “restricted” in Chrome / Edge

It is the X11 port, so browsers block it with **ERR_UNSAFE_PORT**. Fix: run on another port — `PORT=6060 npm start` — or launch Chrome with `--explicitly-allowed-ports=6000`.

2 Open it from your phone on the same Wi-Fi

Find the PC's IPv4 with `ipconfig` (e.g. 192.168.1.20), then browse `http://192.168.1.20:6000`. If blocked, allow the port once as admin: `netsh advfirewall firewall add rule name="Node 6000" dir=in action=allow protocol=TCP localport=6000`

3 No hot reload on the server

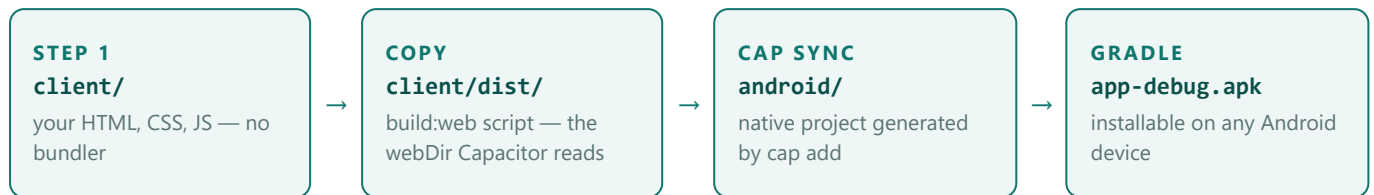
Restart `npm start` after editing **server/**. Client files under **client/** need only a browser refresh.

4 data/school.db is the whole system

Every student, payment and attendance record lives in this single file — back it up regularly and never commit it.

04 Capacitor — Big Picture & One-Time Setup

The toolchain you install once, then forget



ONE-TIME TOOLCHAIN — DO THIS BEFORE TOUCHING CAPACITOR

a) Install a JDK. Easiest: install **Android Studio** (it bundles its own JDK) — developer.android.com/studio. Or standalone **Eclipse Temurin JDK 21** from adoptium.net.

b) Set environment variables (Windows → Environment Variables, or in Git Bash `~/.bashrc`):

```
$ export ANDROID_HOME="C:\\Users\\Ahmed\\AppData\\Local\\Android\\Sdk"
$ export JAVA_HOME="C:\\Program Files\\Eclipse Adoptium\\jdk-21" # adjust to your install
$ export PATH="$PATH:$ANDROID_HOME/platform-tools" # gives you `adb`

# verify — both must answer before anything else works
$ java -version # should print 21.x
$ adb version # should print the SDK version
```

✓ Your SDK is already complete

Platforms and build-tools are installed — after setting the two variables there is nothing left to download from SDK Manager.

✓ First Gradle build needs internet

Gradle downloads itself + dependencies once; after that, builds work fully offline — like the app itself.

05 Wrapping the App for Android

Install · build the web assets · generate · compile

1 — Install Capacitor. The config file already knows the app: `appId: com.school.nidham`, name مدرسة حسين أبوهامد الأهلية, `webDir: client/dist`, `androidScheme: https`.

```
$ cd /d/DEV/School_system
$ npm install @capacitor/core @capacitor/cli @capacitor/android
$ npm install @capacitor/splash-screen @capacitor/haptics @capacitor/filesystem @capacitor/share # optional plugins
```

2 — Copy the web assets into client/dist. There is no bundler, so “build” means *copy*. Save as `scripts/build-web.js` and add `"build:web": "node scripts/build-web.js"` to package.json scripts:

```
// scripts/build-web.js – copy client/ into client/dist/ for Capacitor
const fs = require('fs'), path = require('path');
const src = path.join(__dirname, '..', 'client');
const out = path.join(src, 'dist');
fs.rmSync(out, { recursive: true, force: true });
fs.mkdirSync(out, { recursive: true });
for (const e of ['index.html', 'js', 'screens', 'components', 'styles', 'assets'])
  fs.cpSync(path.join(src, e), path.join(out, e), { recursive: true });
console.log('web assets copied to client/dist');
```

3 — Generate the Android project, 4 — compile the APK.

```
$ npm run build:web          # client/ → client/dist/ (run after every client change)
$ npx cap add android        # generates android/ (first time only)
$ npx cap sync android       # copies dist + plugins into the project
$ npx cap open android       # Android Studio → Build APK(s)
# ...or command line only:
$ cd android && ./gradlew assembleDebug
# APK appears at android/app/build/outputs/apk/debug/app-debug.apk
$ adb install -r app/build/outputs/apk/debug/app-debug.apk
```

THE HABIT THAT SAVES AN HOUR

Capacitor **copies** files, it does not link them. After *any* client change: `npm run build:web` → `npx cap sync android` → rebuild. Forgetting **sync** is the #1 “why isn’t my change in the app” cause.

06 The /api Problem — Read Before Shipping

Why the APK cannot talk to your database as-is

WHAT BREAKS

Inside the APK the WebView loads `client/dist` from `https://localhost` — there is **no Node.js, no Express and no better-sqlite3 in the APK**. Every `fetch('/api/...')` in `client/js/api.js` fails. Pick one of two paths:

Option A — LAN server · ~5 minutes

Phone
APK
WebView

→

PC server
Express
:6000

→

school.db
on the
PC

CONFIG ONLY — SAME CODE

Point the WebView at your dev server over Wi-Fi. Needs the PC on and both devices on the same network; data stays on the PC. Perfect for demos and quick testing.

Option B — On-device SQLite · the real product

Phone
APK
WebView

→

JS data
layer

→

SQLite
on the
phone

DAYS OF WORK — TRUE OFFLINE

Rewrite the data layer onto `@capacitor-community/sqlite` — swap the fetch calls in `api.js` for local queries. No server needed; data lives on the device. This is the end goal for an offline-first school app.

	A — LAN SERVER	B — ON-DEVICE SQLITE
Effort	~5 minutes, config only	Days — rewrite api.js + server logic as client services
Works without PC?	No	Yes
Data location	PC — data/school.db	Phone storage
Best for	Demos, quick testing	The real product

OPTION A SETUP — capacitor.config.ts

```
server: {
  androidScheme: 'https',
  url: 'http://192.168.1.20:6000', // your PC's LAN IP — phone & PC on same Wi-Fi
  cleartext: true,               // required because http:// (not https)
},
```

Then `npm run build:web` → `npx cap sync android` → rebuild. **Remove `url` + `cleartext` before the final build** — or the APK will always point at that one PC.

07 Device Testing & Release

Faster loops while developing, then signing for production

FASTER DEVICE TESTING

1 Wireless ADB — no cable

Android 11+: *Developer options* → *Wireless debugging*
→ *Pair device*, then `adb pair <ip:port>` and `adb connect <ip:port>` .

2 Live reload via Option A

With `server.url` set, editing client files and refreshing the app shows changes immediately — the APK is just a browser aimed at your dev server.

3 See inside the WebView

Logs: `adb logcat | grep -i cap.DOM` +
console: open `chrome://inspect` in desktop
Chrome and pick the device.

4 Splash screen is configured

`launchShowDuration: 1200` is already in the
config — but it is silently ignored unless
`@capacitor/splash-screen` is installed.

RELEASE BUILD — SIGNED APK

```
# one-time: create a keystore — keep it + its passwords safe, losing them means no more updates
$ keytool -genkey -v -keystore school-release.keystore -alias school -keyalg RSA -keysize 2048 -
  validity 10000

# wire it into android/app/build.gradle (signingConfigs + release buildType), then:
$ ./gradlew assembleRelease
# android/app/build/outputs/apk/release/app-release.apk
```

VERSION BUMPS LIVE IN GRADLE, NOT PACKAGE.JSON

Change `versionCode` / `versionName` in `android/app/build.gradle` — the npm version does not propagate to the APK.

08 Tips Cheat-Sheet

The ten things worth remembering

1 Port 6000 → ERR_UNSAFE_PORT

Chrome/Edge block it. Use `PORT=6060 npm start` or the Chrome flag `--explicitly-allowed-ports=6000`.

2 Back up data/school.db

It is the entire system — students, money, attendance. Never commit it to git.

3 Copies, not links

Capacitor snapshots files at sync time. Client change → `npm run build:web` → `cap sync` → rebuild.

4 RTL safe areas

`viewport-fit=cover` is already set; if content hides behind notches, add `env(safe-area-inset-*) padding` in `main.css`.

5 Fonts are bundled

Cairo woff2 ships inside `client/assets` — Arabic renders with zero internet. Keep bundling any new font.

6 better-sqlite3 never runs in the WebView

It is a native Node module — on mobile it is replaced by `@capacitor-community/sqlite` (\$06, Option B).

7 tel: links work as-is

Phone taps already use `tel:` and maps open natively — add `@capacitor/haptics` for attendance tap feedback.

8 Exports on device

CSV/Excel exports need `@capacitor/filesystem` (save to device) + `@capacitor/share` (send to WhatsApp etc.).

9 Chromaflow is your reference

It already has a working android/ build in this workspace — peek at its `launch.md` for a second Capacitor example.

10 Restart, don't assume

Server edits need `npm start` again; client edits need sync + rebuild on mobile, only a refresh in the browser.

09 Troubleshooting & Daily Commands

Symptom → cause → fix, and the two commands you type every day

SYMPTOM	CAUSE → FIX
ERR_UNSAFE_PORT in browser	Port 6000 blocked → <code>PORT=6060 npm start</code>
<code>npx cap add android</code> fails	client/dist missing or empty → run <code>npm run build:web</code> first
SDK location not found	Set <code>ANDROID_HOME</code> , or create android/local.properties with <code>sdk.dir=C:\\Users\\Ahmed\\AppData\\Local\\Android\\Sdk</code>
Unable to locate a Java Runtime	Install JDK 21 and set <code>JAVA_HOME</code>
App opens, every request fails	No /api server inside the APK → §06, pick Option A or B
Change not visible in the APK	Forgot <code>npm run build:web</code> + <code>cap sync</code> before rebuilding
Phone not listed by adb devices	Enable USB debugging, accept the prompt — or pair wirelessly (§07)
APK can't reach <code>http://192.168.x.x</code>	Missing <code>cleartext: true</code> , or firewall blocks port 6000

```
# — browser development —
$ cd /d/DEV/School_system
$ npm start
$ PORT=6060 npm start
$ npm run seed:demo
```

```
# — capacitor / android —
$ npm run build:web
$ npx cap sync android
$ npx cap open android
$ adb logcat | grep -i cap
```

THE 30-SECOND LOOP

Browser: edit → refresh. **Device (Option A):** edit → refresh app. **Device (bundled):** edit → `npm run build:web` → `npx cap sync android` → `gradlew assembleDebug` → `adb install -r`.